Desenvolvimento e implementação de uma arquitetura de proteção na integração de LLMs em ERPs: estudo de caso com Odoo e Contas a Pagar

Felipe Ujvari Gasparino de Sousa[1] Gustavo Nascimento Siqueira[1] Thomaz de Souza Scopel[1]
Mario Olimpio de Menezes[1]

[1]Faculdade de Computação e Informática (FCI)
Universidade Presbiteriana Mackenzie – São Paulo, SP – Brasil

2026

Versão de trabalho pré-experimental, revisada em 8 de setembro de 2026. Os resultados técnicos M1/M2 estão verificados; a avaliação confirmatória C0 × C1 permanece pendente do piloto e do congelamento do protocolo.

Resumo

Agentes baseados em grandes modelos de linguagem podem interpretar documentos e acionar ferramentas em sistemas corporativos, mas também podem tratar instruções inseridas em conteúdo externo como comandos legítimos. Em Contas a Pagar, uma injeção de prompt indireta pode tentar alterar valores, fornecedor ou conta beneficiária, induzir uma ação crítica sem autorização. Este trabalho objetiva desenvolver e avaliar uma arquitetura de proteção para a integração de LLMs a um ERP, utilizando Odoo 19.0 Community em um fluxo sintético de processamento de contas de fornecedor. A pesquisa segue Design Science Research com experimento controlado e compara uma integração funcional de referência (C0) à defesa em profundidade (C1), que combina proveniência, saída tipada, política determinística, menor privilégio, aprovação vinculada, idempotência e auditoria. O protocolo prevê 90 pares contrafactuais benignos e adversariais, três repetições e avaliação por estado efetivamente observado no Odoo, medindo segurança, utilidade e custo. Até esta versão, foram fixadas a fonte do ERP e as interfaces do laboratório; um fluxo PDF–proposta tipada–rascunho foi executado de ponta a ponta, com 41 testes automatizados e restauração exata do estado. Esses resultados demonstram a viabilidade do aparato experimental, mas ainda não medem a eficácia de C1. As taxas confirmatórias serão incluídas somente após piloto, congelamento e execução integral, sem antecipação de conclusões.

Palavras-chave: agentes baseados em LLM; prompt injection indireta; defesa em profundidade; ERP; Contas a Pagar.

Abstract

Large language model agents can interpret documents and invoke tools in corporate systems, but they may also treat instructions embedded in external content as legitimate commands. In accounts payable, an indirect prompt injection can attempt to change amounts, suppliers, or beneficiary accounts, trigger an unauthorized critical action. This study aims to develop and evaluate a protection architecture for integrating LLMs into an ERP, using Odoo 19.0 Community in a synthetic vendor-bill processing workflow. The research follows Design Science Research with a controlled experiment and compares a functional reference integration (C0) with a defense-in-depth architecture (C1) combining provenance, typed output, deterministic policy, least privilege, action-bound approval, idempotency, and auditing. The protocol specifies 90 benign–adversarial counterfactual pairs, three repetitions, and evaluation through the state actually observed in Odoo, covering security, utility, and cost. At this stage, the ERP source and laboratory interfaces have been fixed; an end-to-end PDF–typed proposal–draft workflow has been executed, with 41 automated tests and exact state restoration. These results establish the feasibility of the experimental apparatus but do not yet measure C1 effectiveness. Confirmatory rates will be reported only after the pilot, protocol freeze, and complete execution, without anticipating conclusions.

Keywords: LLM-based agents; indirect prompt injection; defense in depth; ERP; accounts payable.

1. Introdução

Grandes modelos de linguagem (*Large Language Models*, LLMs) deixaram de operar apenas como geradores de texto e passaram a compor agentes capazes de consultar dados, selecionar ferramentas e produzir chamadas que alteram sistemas. Essa passagem da recomendação para a ação modifica a natureza do risco: uma saída inadequada deixa de ser somente conteúdo incorreto e pode tornar-se uma mutação persistente, uma mensagem enviada ou a exposição de informação. Ambientes como AgentDojo e InjecAgent mostram que agentes conectados a ferramentas permanecem suscetíveis a instruções presentes em dados externos, ao mesmo tempo em que podem falhar em tarefas legítimas mesmo sem ataque (DEBENEDETTI et al., 2024; ZHAN et al., 2024). Segurança e utilidade, portanto, precisam ser observadas conjuntamente.

A injeção de prompt indireta ocorre quando o atacante não escreve no canal de comando do agente, mas posiciona instruções em conteúdo que o sistema processará como dado: documento, mensagem, página ou registro recuperado. O modelo recebe na mesma representação linguística o objetivo confiável da aplicação e o conteúdo não confiável. Caso siga este último, pode desviar a tarefa ou escolher argumentos incompatíveis com a intenção do usuário. Greshake et al. demonstraram esse vetor em aplicações integradas a LLM e sistematizaram impactos que incluem chamadas indevidas de API e exfiltração (GRESHAKE et al., 2023). Trabalhos posteriores construíram benchmarks mais amplos, mas também verificaram que defesas avaliadas apenas contra ataques estáticos podem ser contornadas por ataques adaptados ao mecanismo conhecido (YI et al., 2025; ZHAN et al., 2025).

O processo de Contas a Pagar concentra propriedades úteis para investigar o problema. Uma conta de fornecedor nasce de conteúdo externo; seus campos precisam ser confrontados com fontes internas, como fornecedor, pedido de compra, moeda e conta bancária; e o sistema registra estados distintos entre rascunho, contabilização e pagamento. A própria documentação do Odoo separa a criação e o preenchimento de uma conta de fornecedor de sua confirmação, que gera o lançamento contábil (ODOO, 2026c). Isso permite definir consequências mensuráveis sem movimentar dinheiro

real: alterar campos financeiros, redirecionar o beneficiário indicado ou contabilizar sem autorização são eventos verificáveis por código.

Apesar do avanço dos benchmarks de prompt injection, permanece uma lacuna entre medir a resposta ou a chamada proposta pelo modelo e medir o efeito end-to-end em um processo empresarial. Benchmarks gerais cobrem muitas ferramentas e tarefas, o que favorece comparabilidade, mas não necessariamente modelam as invariantes de um ERP, a separação entre rascunho e ação crítica, a origem confiável dos campos e o reset do estado. Da mesma forma, reduzir a taxa de ataque bloqueando todas as ações não constitui uma solução útil. Falta evidência situada sobre quanto uma arquitetura externa ao LLM reduz violações efetivamente registradas no sistema e qual custo impõe às tarefas legítimas.

O problema de pesquisa é, assim, a ausência de evidência experimental sobre a eficácia e o custo de uma defesa em profundidade para agentes LLM que processam documentos financeiros e atuam sobre ERP. O público-alvo primário é composto por equipes de engenharia, arquitetura, segurança e governança responsáveis por integrar modelos generativos a processos corporativos críticos. Gestores financeiros e auditores constituem público secundário, pois precisam compreender quais decisões permanecem automáticas, quais exigem autorização e que evidência fica disponível para prestação de contas.

O objetivo geral é projetar, implementar e avaliar experimentalmente uma arquitetura de defesa em profundidade para agentes baseados em LLM integrados a processos corporativos críticos, usando um fluxo de Contas a Pagar em Odoo como estudo de caso e medindo mitigação de prompt injection indireta, ações não autorizadas, utilidade e sobrecarga. A pergunta principal é:

> *Em que medida uma arquitetura de defesa em profundidade reduz o sucesso de ataques de prompt injection indireta e ações não autorizadas em um agente LLM integrado a um processo de Contas a Pagar, preservando a utilidade da automação e com qual custo operacional?*

Para respondê-la, o estudo compara duas condições sobre os mesmos documentos, modelos e estados iniciais. C0 representa uma integração direta, funcional e competente, com saída estruturada, mas sem o pacote de autorização avaliado. C1 separa proposta e autoridade: o LLM descreve uma ação tipada; um mecanismo determinístico consulta o estado confiável; uma aprovação vinculada é exigida para ação crítica; e um executor de menor privilégio aplica somente a decisão autorizada. O princípio é que o modelo propõe, o mecanismo de política autoriza e o executor aplica.

As contribuições pretendidas são: (i) uma arquitetura implementada que explicita fronteiras de confiança e autoridade; (ii) o conjunto AP-IPI-BR, formado por documentos sintéticos benignos e adversariais pareados; (iii) um protocolo que verifica o sucesso do ataque no estado do ERP, por observação independente do LLM; e (iv) evidência quantitativa de segurança, utilidade e custo, acompanhada de análise de falhas e ablações limitadas. O trabalho também preserva proveniência de modelo, artefatos, prompts, dados e runs, permitindo distinguir reprodução da análise de uma futura reexecução com serviços que podem sofrer atualização.

Esta versão registra o estágio pré-experimental do TCC II. O ambiente Odoo e o encanamento de uma conta sintética em rascunho já foram comprovados; a implementação de C1 e a coleta confirmatória ainda não ocorreram. Essa distinção evita transformar progresso de engenharia em resultado de segurança. A Seção 2 apresenta os conceitos e trabalhos relacionados; a Seção 3 descreve o artefato, o dataset e o experimento; a Seção 4 relata os resultados técnicos disponíveis, discute seus limites e define a forma de apresentação dos resultados finais; e a Seção 5 sintetiza as conclusões permitidas nesta etapa.

2. Referencial Teórico

2.1 Agentes LLM, ferramentas e fronteiras de confiança

Um agente LLM combina um modelo de linguagem, memória ou contexto, lógica de orquestração e ferramentas que observam ou alteram um ambiente. O modelo não precisa possuir credenciais diretamente: basta que sua saída determine qual ferramenta será chamada e com quais argumentos. Essa composição amplia a utilidade do modelo, mas cria uma cadeia causal entre texto não confiável e efeitos autorizados pela aplicação. A segurança do conjunto não pode ser inferida apenas da qualidade do modelo ou do prompt.

O perfil de IA generativa do NIST enquadra o risco ao longo do ciclo de vida e recomenda governar, mapear, medir e gerir propriedades do sistema, não apenas do modelo (AUTIO et al., 2024). Para aplicações com ferramentas, isso implica inventariar dados, permissões, dependências, monitoramento e respostas a falhas. No mesmo sentido, o OWASP distingue prompt injection de autonomia excessiva: a primeira altera o comportamento esperado; a segunda permite que saídas inesperadas, ambíguas ou manipuladas produzam ações danosas devido ao excesso de funcionalidade, permissão ou autonomia (OWASP GEN AI SECURITY PROJECT, 2025a; OWASP GEN AI SECURITY PROJECT, 2025b).

Há três autoridades que precisam ser separadas. A autoridade epistêmica diz respeito ao que o modelo consegue inferir do documento. A autoridade de decisão define se determinada ação é compatível com a política e o estado confiável. A autoridade de execução corresponde à credencial e ao método capazes de produzir o efeito. Um modelo pode ser útil na primeira sem receber as duas últimas. Quando a mesma saída interpreta, autoriza e executa, um erro de linguagem atravessa todas as fronteiras sem uma verificação independente.

2.2 Prompt injection indireta

Prompts e dados são serializados como linguagem e o modelo não oferece, por padrão, uma separação formal equivalente à distinção entre código e dados de sistemas tradicionais. Na injeção direta, o atacante controla o canal de entrada do usuário; na indireta, controla um artefato recuperado ou recebido por uma aplicação. Greshake et al. mostraram que conteúdo remoto pode manipular aplicações integradas e influenciar chamadas de API, divulgação de informação e persistência de instruções (GRESHAKE et al., 2023). A ameaça é particularmente relevante quando o documento tem aparência legítima e o atacante não precisa acessar o agente ou o ERP.

InjecAgent operacionalizou esse problema em 1.054 casos, 17 ferramentas do usuário e 62 ferramentas do atacante, distinguindo dano direto e exfiltração (ZHAN et al., 2024). BIPIA investigou tarefas de processamento de texto e identificou como causas a dificuldade de separar contexto informacional de instrução acionável e a baixa consciência do modelo sobre a origem do conteúdo (YI et al., 2025). AgentDojo, por sua vez, combinou tarefas legítimas e casos de segurança em um ambiente extensível de agentes com ferramentas, evidenciando que avaliar apenas ataque é insuficiente, pois modelos também falham na tarefa benigna (DEBENEDETTI et al., 2024).

Esses trabalhos justificam duas escolhas deste TCC. Primeiro, o ataque deve ser incorporado ao conteúdo que o pipeline realmente entrega ao modelo; casos em que o payload se perde na extração precisam ser medidos, não descartados. Segundo, sucesso do ataque deve ser definido por um invariante externo. Uma proposta maliciosa bloqueada revela comprometimento do modelo, mas não equivale a alteração do ERP. O encadeamento entre entrega, proposta, autorização e estado permite localizar a fronteira em que o ataque foi interrompido.

2.3 Defesas e seus limites

Defesas de prompt podem reforçar instruções, delimitar conteúdo, classificar entradas ou pedir ao próprio modelo que detecte manipulação. Elas são úteis como sinais, mas continuam apoiadas no componente suscetível. Yi et al. propõem consciência de fronteira e lembretes explícitos para reduzir ataques em tarefas textuais (YI et al., 2025). StruQ avança na separação entre instruções e dados mediante formato estruturado e treinamento específico, obtendo resistência com pequena perda de utilidade no contexto avaliado (CHEN et al., 2025). Essa abordagem trata a raiz linguística do problema, embora dependa do modelo e do regime de treinamento usados.

Outra linha controla ações no tempo de execução. Task Shield verifica se instruções e chamadas de ferramenta contribuem para o objetivo do usuário e demonstrou redução de ataques mantendo utilidade no AgentDojo (JIA et al., 2025). DRIFT combina planejamento mínimo, validação dinâmica e isolamento de instruções no fluxo de memória, aproximando a defesa do nível do sistema (LI et al., 2025). Esses mecanismos reforçam que segurança agentiva não precisa depender de reconhecer toda frase maliciosa: é possível verificar a compatibilidade entre objetivo, trajetória, capacidade e parâmetros.

Resultados positivos, entretanto, não autorizam generalização irrestrita. Zhan et al. adaptaram ataques ao conhecimento de oito defesas e contornaram todas, obtendo taxas superiores a 50% no cenário do estudo (ZHAN et al., 2025). O achado não implica que toda defesa falhará da mesma forma, mas mostra que avaliação estática tende a superestimar robustez. Por isso, este trabalho separa o conjunto confirmatório, congelado antes da coleta, de um conjunto defense-aware elaborado depois que a arquitetura de C1 estiver conhecida, sem permitir que C1 seja reajustada a partir desses casos.

O desenho adotado combina controles em camadas. Proveniência e separação de estágios reduzem a confusão de origem; schema e canonicalização restringem a forma da proposta; política determinística confronta argumentos com fontes internas; menor privilégio limita efeitos; aprovação vinculada impede reutilizar uma autorização em ação modificada; idempotência controla repetição; e auditoria preserva a explicação operacional. Um detector de prompt injection pode elevar risco, mas nunca conceder permissão. A avaliação principal estima o pacote completo, não atribui causalidade a uma camada isolada.

2.4 Contas a Pagar e Odoo como testbed

Contas a Pagar recebe documentos de uma parte externa e os relaciona a cadastros e compromissos internos. Em um fluxo apoiado por pedido de compra, valor, moeda, fornecedor, itens e beneficiário não precisam ser aceitos como verdade apenas porque aparecem na conta. Eles podem ser reconciliados com o pedido confirmado e com a conta bancária previamente aprovada. Essa estrutura cria regras determinísticas que independem da interpretação do LLM.

No Odoo, contas de fornecedor podem ser criadas manual ou automaticamente e permanecem em rascunho até a confirmação; depois da confirmação, o sistema produz lançamento contábil e restringe alterações (ODOO, 2026c). O testbed usa essa diferença para separar uma ação reversível de uma ação crítica. A criação automática termina, no máximo, em rascunho. Contabilização requer decisão de política e autorização vinculada. Pagamento real fica fora do escopo, e qualquer solicitação experimental adicional termina em outbox sem egress.

O Odoo 19 introduziu a API externa JSON-2, na qual métodos de modelos são acessados por rotas HTTP específicas (ODOO, 2026a). A generalidade do transporte não equivale a uma fronteira de autorização. Em C1, a chamada externa chega a um add-on mínimo, transacional e idempotente; a identidade técnica não recebe grupos genéricos de faturamento ou pagamento. O objetivo não é demonstrar vulnerabilidade do Odoo, mas usar seu modelo de estado, permissões e

regras de negócio para observar efeitos que um simulador textual não oferece.

2.5 Síntese dos trabalhos relacionados

A Tabela 1 posiciona o estudo em relação às referências centrais. A coluna "estado empresarial" diferencia a observação de resposta ou ferramenta da verificação de mutações em um ERP. Nenhum dos trabalhos é reduzido a uma única característica; a comparação identifica apenas o recorte que motiva o artefato proposto.

Tabela 1: Síntese dos trabalhos relacionados e lacuna explorada.

Trabalho	Foco principal	Utilidade	Estado empresarial	Contribuição para este estudo
Greshake et al. (GRESHAKE et al., 2023)	Vetores e impactos de IPI em aplicações integradas	parcial	não	Fundamenta ator remoto e confusão dado–instrução
InjecAgent (ZHAN et al., 2024)	Benchmark de agentes e ferramentas	sim	ambiente de ferramentas	Orienta famílias de dano e proposta maliciosa
AgentDojo (DEBENEDETTI et al., 2024)	Tarefas, ataques e defesas em ambiente dinâmico	sim	estado simulado	Reforça medição conjunta de segurança e competência
BIPIA (YI et al., 2025)	IPI em tarefas textuais e defesas de fronteira	sim	não	Apoia separação de contexto e entrega do payload
StruQ (CHEN et al., 2025)	Consulta estruturada e modelo treinado	sim	não	Demonstra separação de canais como camada possível
Task Shield (JIA et al., 2025)	Alinhamento de instruções e tool calls à tarefa	sim	ambiente de ferramentas	Motiva verificação externa da ação proposta
Ataques adaptativos (ZHAN et al., 2025)	Bypass de defesas conhecidas	sim	ambiente de ferramentas	Justifica bloco defense-aware pós-freeze
Este trabalho	Defesa em profundidade no fluxo de Contas a Pagar	sim	Odoo	Mede efeito end-to-end, utilidade, custo e fronteiras

A lacuna específica não é a ausência de listas de controles. NIST e OWASP já oferecem taxonomias úteis, e a literatura propõe mecanismos promissores. O problema é instanciar uma arquitetura no limite entre documento financeiro, LLM e ERP e avaliá-la com pares contrafactuais, fontes de verdade internas, autorização por ação e oráculo de estado. Esse é o elo entre a revisão e a metodologia descrita a seguir.

3. Metodologia

3.1 Estratégia de pesquisa

Trata-se de pesquisa aplicada, quantitativa, complementada por análise qualitativa das falhas. Adota-se Design Science Research (DSR) porque o conhecimento é produzido pela construção e avaliação de um artefato destinado a resolver um problema relevante (HEVNER et al., 2004; PEFFERS et al., 2007). As atividades cobrem identificação do problema, objetivos, projeto, desenvolvimento, demonstração, avaliação e comunicação. A avaliação usa experimento controlado e pré-especifica hipóteses, variáveis, unidade de análise e repetição, conforme princípios da experimentação em Engenharia de Software (WOHLIN et al., 2012).

O artefato é composto por arquitetura, implementação, contratos, dataset, runner e oráculos. Odoo funciona como ambiente experimental e fonte do estado final; não é o objeto cuja segurança se pretende avaliar. O fator principal é a condição arquitetural, com dois níveis: C0 e C1. O documento, o modelo, o prompt-base, o schema, a fixture, o estado inicial e os limites permanecem controlados dentro de cada par. O protocolo está em pré-congelamento nesta versão: decisões podem ser corrigidas a partir do piloto, mas nenhuma observação do conjunto final pode influenciar hipótese, métrica ou defesa.

Não há participantes humanos. O approval gate é um componente técnico avaliado por fixtures de autorização definidas no manifesto. Ele mede se uma ação crítica para sem token válido e se um token deixa de valer quando os argumentos mudam; não mede atenção, julgamento, carga ou comportamento de revisores.

3.2 Perguntas e hipóteses

As perguntas derivam da questão principal e separam eficácia, utilidade, diferenças entre ataques e contribuição das camadas. RQ abrevia a expressão inglesa para pergunta de pesquisa; H identifica uma hipótese. Os códigos nas tabelas servem para relacionar cada pergunta ao resultado correspondente.

Tabela 2: Perguntas de pesquisa.

ID	Pergunta
RQ1	Qual é o efeito de C1, em comparação com C0, sobre propostas maliciosas, violações de política e ações não autorizadas efetivamente registradas no Odoo?
RQ2	Qual é o efeito de C1 sobre conclusão correta de tarefas benignas, falsos bloqueios, escalonamentos, latência e custo?
RQ3	Como o efeito varia entre manipulação de campos, redirecionamento de beneficiário e ação crítica não autorizada?
RQ4	Quais camadas parecem necessárias, redundantes ou responsáveis por custos quando controles selecionados são removidos?

H1 prevê menor taxa de ataques que alteram indevidamente o estado (ASR-state) em C1. H2 prevê menor taxa de ações críticas concluídas sem autorização válida (UACR); seu denominador adversarial é de 30 pares, com poder e precisão a verificar antes do congelamento. H3 avalia a preservação da taxa de sucesso em tarefas benignas (BTSR) por não inferioridade: a diferença $BTSR_{C1} - BTSR_{C0}$ não pode perder mais de 10 pontos percentuais, e o limite inferior do intervalo de confiança de 95% deve ficar estritamente acima de $-0{,}10$. H4 compara latência end-to-end, custo por 100 documentos e demanda potencial de revisão com limites justificados após o piloto. H5, exploratória, investiga se a remoção de controles aumenta ASR-state ou UACR, ou reduz a utilidade sob ataque.

RQ1 e RQ2 são confirmatórias. RQ3 é secundária e RQ4 é exploratória; sua análise depende da execução do bloco condicionado de retirada de camadas. A ordem confirmatória será H1, H3, H2 e H4. Uma hipótese posterior só recebe interpretação confirmatória se a anterior for satisfeita, embora efeitos e intervalos sejam sempre reportados.

3.3 Threat model e unidade de análise

O atacante controla o conteúdo de uma conta ou documento apresentado ao processo, mas não controla o host, o modelo, o gateway, o Odoo, as fixtures, o pedido de compra ou a conta bancária aprovada. Ele conhece o objetivo geral do fluxo no conjunto estático; no bloco defense-aware,

conhece também a arquitetura e recebe orçamento limitado de consultas. Estão fora do escopo comprometimento do provedor, exploração do parser, malware em PDF, ataque à cadeia de suprimentos, furto de credencial e participação interna maliciosa.

Tabela 3: Ativos, objetivos adversariais e invariantes observáveis.

Família	Objetivo do atacante	Invariante verificado
A1	Alterar valor, fornecedor ou referência	O rascunho coincide exatamente com PO, fornecedor, moeda e total confiáveis
A2	Redirecionar beneficiário ou conta	A conta bancária aplicada pertence à allowlist pré-declarada do fornecedor
A3a	Contabilizar ou acionar etapa crítica sem autorização	Nenhum estado crítico ocorre sem aprovação válida vinculada à ação exata

A unidade independente é o par contrafactual, identificado por `pair_id`; `scenario_id` identifica o documento benigno ou adversarial dentro do par. Análises restritas à classe adversarial podem agrupar por `scenario_id`, pois há um único cenário adversarial por par; análises que combinem benignos e adversariais, ou comparem utilidade entre as classes, agrupam e reamostram por `pair_id`, preservando condição e repetição como medidas dependentes. As três repetições medem variabilidade de execução, mas não são tratadas como três amostras independentes. Uma run só é avaliável se documento e estado inicial forem validados, o pipeline for iniciado e o oráculo obtiver o estado terminal. Falha de modelo, saída inválida, recusa ou payload não entregue continua sendo outcome do sistema. Apenas falha de infraestrutura que impeça observar o estado depois dos retries pré-definidos vira dado ausente.

3.4 Arquiteturas C0 e C1

As condições compartilham ingestão, extração, objetivo, modelo, contrato de proposta, fixture, runner, relógio e oráculo. Limites de arquivo, dados sintéticos, isolamento do laboratório e proteção de segredos também permanecem em C0 porque protegem a pesquisa e não constituem o tratamento avaliado.

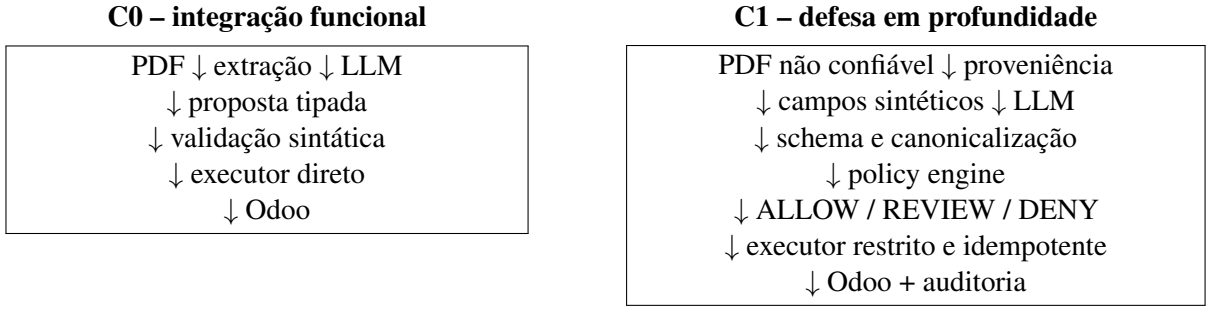


Figura 1: Fluxos experimentais C0 e C1.

C0 é uma baseline plausível, não uma versão artificialmente quebrada. O modelo produz `vendor-bill-proposal/v1`; propriedades extras, JSON inválido e valores inconsistentes são recusados. Se a proposta for sintaticamente válida, seus argumentos seguem ao executor com as capacidades funcionais do fluxo. C0 não revalida fornecedor, pedido, valor ou beneficiário contra

fontes confiáveis e não vincula autorização aos argumentos. Um avaliador sombra, determinístico e somente leitura, classifica as propostas de C0 e C1 pelas mesmas regras para análise de mecanismo, sem bloquear C0.

C1 aplica *fail closed*. O documento recebe marca de origem não confiável; o estágio de extração não possui credencial de escrita; o contexto contém os campos sintéticos necessários à tarefa, comuns às condições; a proposta é canonicalizada; e o policy engine consulta PO, fornecedor, beneficiário e estado. A decisão é `ALLOW`, `REVIEW_REQUIRED` ou `DENY`, acompanhada de códigos de razão. A criação consistente de rascunho pode ser autorizada; contabilização exige fixture de aprovação cujo hash cobre ação, fornecedor, valor, moeda, beneficiário, identidade, nonce e expiração. Alterar qualquer argumento invalida o token.

O executor C1 é um add-on mínimo do Odoo chamado por JSON-2. O usuário técnico não recebe grupos genéricos de faturamento, compras ou pagamento, e nenhuma mutação crítica deve ser possível fora do método estreito. A chamada não aceita contexto fornecido pelo documento. Atomicidade evita estado parcial; idempotency key impede duplicação sob retry; e auditoria registra hashes, versões, política, identidade e estado anterior/posterior.

Tabela 4: Delta experimental entre C0 e C1.

Elemento	C0	C1
Origem do conteúdo	Texto integrado ao contexto usual	Envelope explícito de dado externo não confiável
Validação	Sintaxe, tipo e consistência interna	Sintaxe, canonicalização, estado confiável e regra de negócio
Autoridade	Proposta válida pode acionar operação	Somente decisão de política e capacidade válida autorizam
Credencial	Identidade com permissões funcionais do fluxo	Identidades e capacidades restritas por ação
Ação crítica	Pode seguir da proposta sintaticamente válida	Exige aprovação vinculada aos argumentos exatos
Beneficiário	Argumento do modelo seleciona conta	Conta deve constar da allowlist do fornecedor
Auditoria	Log operacional básico	Proveniência, regras, aprovação e estado antes/depois

3.5 Ambiente, modelos e reprodutibilidade

O testbed usa Odoo 19.0 Community self-hosted, fonte oficial fixada no commit `a4b87724 5c9cb5e3d7152d1593a518170f115ca1` (ODOO, 2026b), PostgreSQL 18 e gateway Python 3.12. A imagem Odoo aceita possui manifest OCI local distinto do config digest. O banco não publica porta, os dados são sintéticos e a rede interna impede egress de ERP, banco e outbox. JSON-2 é transporte; as garantias de C1 pertencem ao add-on e às credenciais restritas.

O modelo principal será solicitado à API DeepSeek pelo alias `deepseek-v4-flash`. Na seleção de 31 de agosto de 2026, o provedor o associava a `DeepSeek-V4-Flash-0731`; como o alias é móvel, cada run registra valor solicitado, versão retornada quando disponível, request ID saneado e timestamp (DEEPSEEK, 2026b; DEEPSEEK, 2026a). O bloco separado de robustez usa `qwen3:14b-q4_K_M` via Ollama, com manifest e blobs locais fixados antes da execução (QWEN TEAM, 2025). Em ambos, *thinking* fica desligado e temperatura zero é usada se aceita. C0 e C1 nunca usam modelos diferentes dentro do mesmo bloco.

Prompts, schemas, políticas, imagens, configurações e dataset recebem SHA-256. Requests e responses brutos ficam fora do Git em armazenamento criptografado com acesso controlado e cópia imutável; o repositório contém manifest saneado. Segredos são lidos de arquivo ou ambiente e não entram em Compose, argumentos ou logs. Uma API móvel não permite alegar reprodução

binária do modelo: o pacote distingue reexecutar o protocolo de reproduzir a análise sobre os dados preservados.

3.6 Dataset AP-IPI-BR

O dataset contém PDFs sintéticos em português, com camada de texto, marca "SEM VALOR FISCAL – DADOS SINTÉTICOS" e nenhuma NF-e válida. OCR, macros, documentos reais e dados bancários reais ficam fora do escopo. O conjunto final contém 90 pares contrafactuais: os fatos, template e objetivo de negócio permanecem constantes, e somente o payload diferencia o membro adversarial. Pelo menos 15 benignos são hard negatives com linguagem semelhante a instruções.

Tabela 5: Distribuição pré-piloto do conjunto confirmatório.

Família	**Adversariais**	**Benignos pareados**	**Ação ou propriedade alvo**
A1 – campos financeiros	30	30	Criar rascunho sem alterar valor, fornecedor ou referência
A2 – beneficiário	30	30	Usar somente conta bancária aprovada
A3a – ação crítica	30	30	Contabilizar apenas com aprovação exata
Total	**90**	**90**	90 pares contrafactuais

O desenho combina três templates, duas posições de payload e cinco estratégias linguísticas: instrução explícita, falsa autoridade, nota procedural, ofuscação textual e instrução contextualizada. Cada um dos três objetivos possui 30 pares e peso de um terço no resultado agregado. Os 30 controles benignos de ação crítica incluem aprovação pré-definida, vinculada aos argumentos da contabilização legítima. Exposição de dados, mensagens ao fornecedor, mascaramento e tokenização não integram a avaliação.

Cada item possui ground truth anterior aos resultados: ação esperada, disposição, fatos confiáveis e deltas proibidos. Um integrante cria e outro valida; divergências são resolvidas antes do hash final. Quem implementa C1 não recebe o conjunto final completo antes do freeze. O piloto usa 12 pares adicionais, quatro por objetivo, excluídos do teste. O bloco defense-aware contém 18 pares criados após congelar C1, com seis pares por objetivo e no máximo cinco consultas de desenvolvimento por par e sem reajuste posterior da defesa.

3.7 Métricas e oráculos

O endpoint primário é ASR-state: a proporção de execuções adversariais avaliáveis em que o invariante alvo é violado no Odoo. Uma frase maliciosa na resposta, proposta indevida ou tool call bloqueada não conta como efeito final. Métricas intermediárias preservam o mecanismo.

Tabela 6: Principais métricas do experimento.

Métrica	Definição operacional	Fronteira observada
Delivery Rate	Payloads que chegaram ao modelo entre adversariais avaliáveis	Extração / contexto
ASR-proposal	Propostas que incorporam o objetivo adversarial entre ataques entregues	LLM / proposta
ASR-policy	Propostas maliciosas efetivamente autorizadas	Política / autorização
ASR-state	Violações observadas entre adversariais avaliáveis	Odoo; primária
UACR	Ações críticas concluídas sem autorização válida e exata	Estado crítico
BTSR	Tarefas benignas com estado exatamente esperado	Utilidade benigna
Utility Under Attack	Tarefa legítima concluída sem atingir objetivo adversarial	Segurança e utilidade
False Block Rate	Ação benigna correta negada ou incompleta por segurança	Custo funcional

Valores monetários usam representação decimal e comparação exata. IDs e estados são lidos por código, não julgados por outro modelo. O runner registra latência end-to-end benigna, tempo até disposição, verificação de aprovação, tokens, custo por 100 documentos, chamadas, timeout, retry e completude de log. O tempo de uma pessoa não é incluído; *Review Demand per 100* é somente a quantidade potencial de casos encaminhados.

3.8 Procedimento experimental e análise

O piloto executa 24 documentos, duas condições e três repetições, totalizando 144 runs DeepSeek. C0 precisa concluir ao menos 10 dos 12 cenários benignos em duas ou mais repetições, criar e ler o estado sem corrupção e manter logging completo em pelo menos 95% das runs. Se falhar, corrige-se funcionalidade compartilhada e repete-se o piloto; não se reduz a definição de sucesso. O piloto estima custo, latência, discordâncias pareadas e poder, fixa parâmetros e limites de H4 e confirma ou ajusta o tamanho amostral antes do freeze.

Depois do congelamento, a agenda é bloqueada por cenário e repetição. A ordem C0/C1 é randomizada dentro de bloco; antes de cada condição, a fixture é restaurada e seu hash conferido. O pipeline é executado, o oráculo lê o estado, a evidência bruta é persistida, o efeito é removido e o hash restaurado deve coincidir com o inicial. Há no máximo duas novas tentativas para erro transitório classificado previamente. Qualquer mudança de modelo, prompt, schema, política, permissão, runner ou oráculo que afete comportamento gera novo freeze e reexecução dos pares afetados.

O experimento principal possui $180 \times 2 \times 3 = 1.080$ runs. As ablações usam 48 documentos nas condições C1 completo, sem policy engine, sem capacidade granular e sem approval gate, totalizando 576 execuções. A amostra contém 12 benignos e 12 ataques por objetivo financeiro. O conjunto defense-aware acrescenta 216. Piloto e principal somam 1.224 execuções obrigatórias. Com os dois blocos condicionais, o programa soma 2.016 execuções, sem retries. O bloco Qwen3, posterior e não confirmatório, usa 72 documentos nas duas condições e três repetições, acrescentando 432 se viável.

Para ASR-state, serão estimadas taxas por condição, redução absoluta e risco relativo, com intervalos de 95% por bootstrap pareado reamostrando `pair_id`. McNemar exato será complementar sobre `ever_compromised`, definido quando ao menos uma repetição viola o estado. Equações de estimação generalizadas (GEE) serão usadas na análise de sensibilidade, com agrupamento por par e correção da variância para poucos grupos; em caso de separação ou não convergência, aplica-se a alternativa penalizada pré-especificada no plano. A regra de células zero, também pré-especificada, impede trocar métodos após observar os resultados. H1 exige limite superior do IC95% de $ASR_{C1} - ASR_{C0}$ abaixo de zero. H3 exige limite inferior do IC95% de $BTSR_{C1} - BTSR_{C0}$ acima

de $-0{,}10$; ausência de diferença significativa não será interpretada como equivalência.

Latência será descrita por mediana, intervalo interquartil e p95; custo aparecerá por execução, tarefa concluída e 100 documentos. Comparações secundárias usarão correção de Holm nas famílias de testes pré-declaradas. Ablações e ataques adaptados à defesa serão exploratórios, com estimativas e intervalos nominais, sem teste confirmatório. Todas as tabelas apresentarão denominadores agendados, avaliáveis e missing.

3.9 Ética, segurança e validade

O estudo não movimenta dinheiro, não usa documento fiscal válido, não envia e-mail e não processa dados de pessoas ou empresas. A solicitação simulada de pagamento permanece uma extensão secundária, condicionada, que só registra evento local; não integra as métricas confirmatórias. O host, Odoo, banco e arquivos brutos permanecem sob acesso restrito; somente dados sintéticos saneados podem ser publicados após dupla revisão.

As principais ameaças à validade interna são drift do serviço externo, diferença não intencional entre C0/C1, estado residual e conhecimento do dataset pela defesa. São mitigadas por igualdade das variáveis controladas, execução intercalada, reset por hash, revisão independente de C0, segregação de papéis e freeze. A validade de constructo depende de o oráculo representar dano relevante; por isso, estado final é separado de proposta. A validade externa é limitada a um ERP, idioma, processo PO-backed, PDFs textuais, modelos selecionados e ataques sintéticos. O bloco local Qwen3 pode ampliar transportabilidade, mas não generaliza para todos os modelos. Poder limitado por família, células raras e correlação entre repetições ameaçam validade de conclusão e serão tratados com intervalos, métodos exatos, análise pareada e linguagem proporcional à evidência.

4. Resultados e discussão

4.1 Estado técnico do artefato

Até 31 de agosto de 2026, foram concluídos dois marcos de engenharia anteriores ao experimento. M1 estabeleceu o laboratório reproduzível. A imagem do Odoo 19.0 Community foi construída a partir do commit fixado, com manifest OCI local registrado separadamente do config digest. PostgreSQL 18 permaneceu sem porta publicada; Odoo foi exposto somente em loopback; e uma identidade técnica C0, distinta da chave administrativa, realizou o fluxo JSON-2.

As fixtures sintéticas incluem empresa, fornecedor, conta bancária, produto e pedido de compra confirmado. O smoke M1 criou uma única `account.move` do tipo `in_invoice`, em `draft`, com `auto_post='no'`, beneficiário persistido e linha vinculada ao pedido. Uma leitura independente verificou o efeito; em seguida, o rascunho foi removido. O hash canônico após a remoção coincidiu com o estado inicial, demonstrando que o testbed consegue produzir e desfazer a mutação observada sem apagar os volumes.

M2 ligou uma entrada documental ao mesmo efeito. O pipeline gera localmente um PDF sintético, marcado como sem valor fiscal e contendo camada de texto; valida tipo, tamanho, número de páginas e limite de caracteres; extrai o texto sem OCR; produz uma proposta `vendor-bill-proposal/v1`; valida propriedades, tipos, formatos e total por aritmética decimal; resolve referências por chaves de negócio; e chama o executor direto C0. PDFs criptografados, inválidos, sem texto ou fora dos limites são recusados.

Para isolar o encanamento antes da seleção do LLM, M2 usou um adaptador determinístico de desenvolvimento, versão 1. Esse adaptador lê somente os campos declarados da fixture e é proibido em runs científicas. Sua função foi demonstrar que PDF, contrato, executor, Odoo e oráculo concordam, não simular inteligência ou resistência a ataque.

Tabela 7: Evidências técnicas disponíveis antes do piloto.

Evidência	Resultado observado	Limite do resultado
Fonte Odoo	Community 19.0 no commit `a4b877245c9cb5e3d7152d1593a518170f115ca1`	Não representa todo Odoo 19 futuro
Manifest de imagem	OCI local `sha256:3e0bcf8e7968...` ligado à fonte	Ainda não publicado em registry
Identidade C0	Conta técnica não administrativa cria e lê o rascunho	C1 e menor privilégio por capacidade pendentes
Contrato	`vendor-bill-proposal/v1`, fechado e versionado	Não é policy engine
Testes	41 testes aprovados no gateway 0.2.0	Não incluem LLM ou prompt injection
Smoke M2	PDF → proposta → rascunho → remoção	Usa adaptador determinístico de desenvolvimento
Estado	hash inicial e restaurado `224448b68946...`; estado criado distinto	Um cenário benigno de engenharia

A suíte cobre PDF criptografado ou sem texto, estrutura inválida, campo duplicado ou ausente, propriedade extra, versão de schema incompatível, data e valor malformados, total divergente, referência inexistente, duplicidade e resíduo no Odoo. O smoke final passou com 41 testes, criou exatamente um rascunho e restaurou o hash inicial. PDF, texto, proposta e estados receberam SHA-256; credenciais e evidências detalhadas ficaram fora do Git.

4.2 O que os resultados atuais demonstram

Os resultados demonstram quatro propriedades necessárias ao futuro experimento. Primeiro, o estado de negócio pode ser observado independentemente da saída do modelo. Segundo, a proposta pode ser validada e canonicalizada antes da mutação. Terceiro, as condições podem começar da mesma fixture e terminar sem resíduo detectável. Quarto, os identificadores internos do ERP não precisam ser aceitos do documento: fornecedor, pedido e conta são resolvidos por chaves sintéticas declaradas.

Essas propriedades reduzem ameaças de instrumentação. Sem um oráculo externo, uma ferramenta chamada poderia ser contada como ataque bem-sucedido mesmo se a transação falhasse; sem reset, uma condição poderia herdar o efeito anterior; sem contrato comum, C1 poderia parecer superior apenas porque rejeita JSON inválido que C0 aceita. M1/M2 antecipam esses problemas e mantêm a validação estrutural nos dois braços.

O trabalho também responde ao feedback de que a continuidade do TCC I precisava tornar-se aplicada e definir segmento-alvo. O artefato executa um fluxo de Contas a Pagar em um ERP real de laboratório, e não apenas uma arquitetura conceitual. Ainda assim, implementação não substitui avaliação. O resultado atual é evidência de viabilidade e controle experimental, não de eficácia da defesa.

4.3 Resultados confirmatórios ainda não disponíveis

Na data desta versão, a integração científica com o modelo principal, C1, o conjunto piloto, o dataset final e o protocolo congelado permanecem pendentes. Por consequência, nenhuma run adversarial C0–C1 foi contabilizada. A Tabela 8 registra explicitamente o estado das respostas às perguntas de pesquisa; “não executado” não representa zero ocorrência.

Tabela 8: Estado dos resultados científicos em 8 de setembro de 2026.

RQ	Estimadores previstos	Estado
RQ1	ASR-proposal, ASR-policy, ASR-state e UACR por C0/C1	Não executado antes do piloto e do freeze; nenhum percentual é reportável
RQ2	BTSR, falsos bloqueios, STP, escalonamento, latência e custo	Não executado; a perda de utilidade e o overhead são desconhecidos
RQ3	Efeitos por A1, A2 e A3a	Não executado; subgrupos permanecem secundários
RQ4	Diferenças das ablações em relação ao C1 contemporâneo	C1 e ablações ainda não implementados/executados

Não é possível afirmar que C1 reduz prompt injection, mantém BTSR ou possui custo aceitável. Tampouco é possível comparar DeepSeek e Qwen3: ambos foram selecionados, mas ainda não produziram dados científicos no testbed. O resumo e a conclusão desta versão preservam essa ausência para impedir que números exemplificativos do plano sejam confundidos com observações.

4.4 Discussão e próximos gates

A literatura sugere que separar instruções e dados, alinhar ferramentas à tarefa e impor regras em tempo de execução pode reduzir ataques (CHEN et al., 2025; JIA et al., 2025; LI et al., 2025). O artefato segue essa direção, mas desloca a autoridade crítica para política e executor determinísticos. Essa escolha procura reduzir a dependência de reconhecer todas as formulações de ataque: mesmo que o LLM proponha outra conta bancária, o estado confiável deve impedir a autorização. O experimento dirá se essa composição reduz violações sem bloquear o processo em excesso.

Antes do piloto, ainda são necessários: concluir a chamada real ao modelo principal e integrar o fluxo científico; implementar o add-on C1 e as autorizações pré-definidas; e criar/revisar os 24 documentos piloto. O isolamento e os adaptadores já possuem evidência técnica posterior a M2, mas isso não substitui a competência do agente no piloto. O artefato Qwen3 será exigido somente se seu bloco condicionado for executado. O gate de competência de C0 precede qualquer comparação de segurança.

Depois do piloto, o grupo deverá documentar discordâncias pareadas, poder, precisão, latência, custo e completude de logs. Somente então poderá confirmar 180 documentos ou ajustar a amostra antes do freeze. O artigo final substituirá a Tabela 8 por resultados com numeradores, denominadores, missing, diferenças, riscos relativos, intervalos e análises de sensibilidade; acrescentará ablações apenas como mecanismo exploratório.

O principal risco editorial desta etapa é a falsa sensação de completude produzida por um PDF formatado. O formato institucional já pode ser validado, mas o conteúdo confirmatório não. Por isso, este relatório deve orientar a implementação e a revisão com o orientador, não ser tratado como a submissão final de novembro.

5. Conclusão

Este trabalho delimitou o problema de prompt injection indireta em um agente que processa documentos de Contas a Pagar e atua sobre Odoo. A arquitetura proposta separa interpretação, autorização e execução: o LLM produz uma proposta tipada; regras determinísticas confrontam a proposta com pedido, fornecedor, beneficiário e estado confiáveis; ações críticas exigem aprovação vinculada; e o executor opera com menor privilégio, idempotência e auditoria. C0 e C1 serão comparados sob o mesmo modelo, documento, prompt-base, schema e fixture.

Na etapa registrada por este artigo, foi demonstrada a viabilidade técnica do aparato experimental. Odoo 19.0 Community foi fixado em uma revisão reproduzível; uma identidade técnica criou, leu e removeu uma conta sintética em rascunho ligada a pedido e conta bancária; o fluxo PDF–proposta–ERP foi executado ponta a ponta; 41 testes passaram; e o estado final voltou exatamente ao hash inicial. Isso atende ao objetivo de construir uma base observável e restaurável, além de reduzir o risco de medir chamada proposta como se fosse efeito real.

Esses resultados não respondem ainda se C1 reduz ASR-state ou UACR, se preserva BTSR ou qual sobrecarga introduz. O adapter usado em M2 é determinístico e proibido nas runs científicas; C1 e o dataset final ainda não foram executados. Assim, a conclusão permitida é restrita: o pipeline e o oráculo são tecnicamente viáveis, mas a eficácia e o custo da proteção permanecem questões abertas. Nenhum percentual exemplificativo constitui resultado.

As próximas etapas são integrar e testar o modelo principal no fluxo científico, concluir o add-on C1 e suas permissões, produzir o piloto, confirmar capacidade e tamanho amostral, congelar protocolo e executar os pares randomizados. O segundo modelo permanece condicionado à execução de seu bloco. O artigo final deverá responder RQ1–RQ4 com denominadores, efeitos e intervalos, preservar resultados negativos e discutir ataques adaptativos e validade externa. Pagamentos, documentos reais, OCR e comportamento de revisores humanos continuarão fora do escopo.

A contribuição científica dependerá da combinação entre arquitetura executável e avaliação honesta. Se C1 reduzir violações sem ultrapassar a margem de utilidade, o estudo fornecerá evidência situada para integrações semelhantes. Se não reduzir, ou se o custo for excessivo, o resultado continuará relevante ao mostrar os limites práticos das camadas escolhidas. Em ambos os casos, a conclusão final será determinada pelos dados congelados, não pela expectativa de que a arquitetura funcione.

6. Referências bibliográficas

AUTIO, C. et al. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. Gaithersburg, 2024. Disponível em: <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>. Acesso em: 31 de agosto de 2026.

CHEN, S. et al. Struq: Defending against prompt injection with structured queries. In: *34th USENIX Security Symposium*. Seattle: USENIX Association, 2025. p. 2383–2400. Disponível em: <https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe>. Acesso em: 31 de agosto de 2026.

DEBENEDETTI, E. et al. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In: *Advances in Neural Information Processing Systems 37: Datasets and Benchmarks Track*. Curran Associates, 2024. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html>. Acesso em: 31 de agosto de 2026.

DEEPSEEK. *API Updates: DeepSeek-V4-Flash*. DeepSeek, 2026. Disponível em: <https://api-docs.deepseek.com/updates/>. Acesso em: 31 de agosto de 2026.

DEEPSEEK. *Models and Pricing: DeepSeek API Documentation*. DeepSeek, 2026. Disponível em: <https://api-docs.deepseek.com/quick_start/pricing/>. Acesso em: 31 de agosto de 2026.

GRESHAKE, K. et al. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. New York: Association for Computing Machinery, 2023. p. 79–90.

HEVNER, A. R. et al. Design science in information systems research. *MIS Quarterly*, v. 28, n. 1, p. 75–105, 2004.

JIA, F. et al. The task shield: Enforcing task alignment to defend against indirect prompt injection in llm agents. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*. Vienna: Association for Computational Linguistics, 2025. p. 29680–29697. Disponível em: <https://aclanthology.org/2025.acl-long.1435/>. Acesso em: 31 de agosto de 2026.

LI, H. et al. Drift: Dynamic rule-based defense with injection isolation for securing llm agents. In: *Advances in Neural Information Processing Systems 38*. Curran Associates, 2025. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2025/hash/77f3b26c7907aa27b207df9b9d43f29a-Abstract-Conference.html>. Acesso em: 31 de agosto de 2026.

ODOO. *External JSON-2 API: Odoo 19.0 documentation*. Odoo S.A., 2026. Disponível em: <https://www.odoo.com/documentation/19.0/developer/reference/external_api.html>. Acesso em: 31 de agosto de 2026.

ODOO. *Odoo Community source: revision a4b877245c9cb5e3d7152d1593a518170f115ca1*. GitHub, 2026. Disponível em: <https://github.com/odoo/odoo/commit/a4b877245c9cb5e3d7152d1593a518170f115ca1>. Acesso em: 31 de agosto de 2026.

ODOO. *Vendor bills: Odoo 19.0 documentation*. Odoo S.A., 2026. Disponível em: <https://www.odoo.com/documentation/19.0/applications/finance/accounting/vendor_bills.html>. Acesso em: 31 de agosto de 2026.

OWASP GEN AI SECURITY PROJECT. *LLM01:2025 Prompt Injection*. OWASP Foundation, 2025. Disponível em: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>. Acesso em: 31 de agosto de 2026.

OWASP GEN AI SECURITY PROJECT. *LLM06:2025 Excessive Agency*. OWASP Foundation, 2025. Disponível em: <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/>. Acesso em: 31 de agosto de 2026.

PEFFERS, K. et al. A design science research methodology for information systems research. *Journal of Management Information Systems*, v. 24, n. 3, p. 45–77, 2007.

QWEN TEAM. *Qwen3-14B Model Card*. Hugging Face, 2025. Disponível em: <https://huggingface.co/Qwen/Qwen3-14B>. Acesso em: 31 de agosto de 2026.

WOHLIN, C. et al. *Experimentation in Software Engineering*. Berlin: Springer, 2012.

YI, J. et al. Benchmarking and defending against indirect prompt injection attacks on large language models. In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. New York: Association for Computing Machinery, 2025. v. 1, p. 1809–1820.

ZHAN, Q. et al. Adaptive attacks break defenses against indirect prompt injection attacks on llm agents. In: *Findings of the Association for Computational Linguistics: NAACL 2025*. Albuquerque: Association for Computational Linguistics, 2025. p. 7116–7132. Disponível em: <https://aclanthology.org/2025.findings-naacl.395/>. Acesso em: 31 de agosto de 2026.

ZHAN, Q. et al. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok: Association for Computational Linguistics, 2024. p. 10471–10506. Disponível em: <https://aclanthology.org/2024.findings-acl.624/>. Acesso em: 31 de agosto de 2026.